Resume Intelligence Platform

Comprehensive Production Architecture, AI Engineering Pipeline & Complete Implementation Roadmap

Target System: Production AI Resume Screener & Recruiter Engine	**Date:** August 2026
Architecture: Monorepo (Next.js 14 + NestJS + Redis/BullMQ + Python AI)	**Database:** PostgreSQL + pgvector

1. Executive Summary & Core Value Proposition

The **Resume Intelligence Platform** is an enterprise-grade AI hiring platform that elevates resume parsing from a basic keyword-matching script into an intelligent talent evaluation engine. Rather than relying on rigid, pre-defined skill whitelists, it dynamically extracts candidate attributes, computes explainable match scores with evidence, evaluates ATS compatibility with actionable diagnostics, and renders executive company-ready suggestion templates.

2. System Architecture & Monorepo Topology

The platform adopts a high-performance modular monorepo structure designed for clean separation of concerns and independent scalability:

```
resume-intelligence/
■■■ apps/
■ ■■■ web/ # Next.js 14 Frontend (App Router, Tailwind, shadcn/ui)
■ ■■■ api/ # NestJS Core API (Auth, RBAC, DB CRUD, Queue Producer)
■ ■■■ worker/ # BullMQ Background Worker (Document pipeline execution)
■■■ services/ai/ # Python FastAPI Microservice (NLP, LLM, Embeddings, SpaCy)
■■■ packages/ # Shared packages (types, validation/zod schemas, config)
■■■ infrastructure/ # Docker, Nginx, PostgreSQL + pgvector, Redis
```

3. Core Subsystems & Technical Pipelines

3.1 Universal Open-Ended Skill Extraction

No Hardcoded Whitelists: Traditional parsers fail when encountering modern, niche, or domain-specific skills (e.g. *'Temporal, LangGraph, OpenTelemetry, RAGAS'*). The system utilizes a multi-tiered pipeline: Candidate Phrase Detection → Semantic LLM Extraction → Canonical Normalization (e.g. 'React.js / ReactJS' → 'React.js') → Dynamic Categorization → Confidence & Evidence Binding.

3.2 Explainable Job Matching Engine

Scores are not opaque black boxes. Every match score includes an explicit breakdown and concrete evidence linking:

- **Composite Scoring Formula:** Overall = (Skills * 0.40) + (Experience * 0.25) + (Projects * 0.20) + (Education * 0.05) + (Semantic Sim * 0.10)
- **Evidence Traceability:** Associates each matched skill with specific projects or roles in the candidate's history.

3.3 ATS Compatibility Engine

Evaluates parsing compliance, section hierarchy, quantified achievement metrics (X-Y-Z formula), job title alignment, and provides high-impact recommendations.

3.4 Grounded AI Resume Suggestions & Company Templates

Strictly grounded in verified candidate experience without fabricating credentials. Generates standardized company hiring templates: *Recruiter Screening Summaries, Interview Focus Checklists, Shortlist Memos, and Constructive Feedback Letters.*

4. Relational & Vector Database Schema (PostgreSQL + pgvector)

Table	Primary Fields	Description
users	id, name, email, password_hash, role	Candidates, Recruiters, and Admins
resumes	id, user_id, file_url, status, file_hash	Raw file tracking & processing state machine
candidate_profiles	id, resume_id, name, email, phone, summary, exp	Parsed candidate structured information
skills	id, canonical_name, category, embedding	Global catalog of normalized skills with vector embeddings
candidate_skills	id, candidate_id, skill_id, confidence, source, evidence	Skill associations with confidence score & provenance
jobs	id, organization_id, title, description, exp_min, exp_max	Target job description requirements
job_skills	id, job_id, skill_id, importance, required	Required vs preferred skills for role
matches	id, candidate_id, job_id, overall_score, breakdown	Detailed evaluation scores
match_evidence	id, match_id, skill_id, candidate_evidence, reason	Explainability layer for auditability
templates	id, name, type, content, variables	Standardized hiring templates

5. REST API Specifications (/api/v1)

- POST /api/v1/resumes/upload - Accepts multipart PDF/DOCX, creates async BullMQ job (returns 202 Accepted).
- GET /api/v1/resumes/:id/status - Polls processing state (QUEUED → EXTRACTING → SKILL_ANALYSIS → COMPLETED).
- GET /api/v1/candidates/:id/profile - Retrieves structured profile, contact info, and categorized skills.
- POST /api/v1/jobs & POST /api/v1/matches - Creates target roles and triggers matching evaluation.
- POST /api/v1/templates/:id/generate - Renders hiring recommendation documents populated with candidate intelligence.

6. 12-Stage Implementation Roadmap

Phase	Milestone Focus	Key Deliverables
Phase 1	Monorepo & Infra	Docker (Postgres + Redis), Next.js, NestJS, Python FastAPI, Turborepo
Phase 2	Data Layer & Auth	Prisma migrations, JWT auth, RBAC (Candidate/Recruiter), S3 upload storage
Phase 3	Async Ingestion Worker	BullMQ queue, PDF/DOCX text extraction, layout & section boundary detection
Phase 4	Universal Skill Engine	Dynamic LLM/SpaCy extraction, canonical mapping, category tagging & confidence
Phase 5	ATS Diagnostic Engine	Section completeness, bullet impact scoring, keyword density analysis
Phase 6	JD Parser & Matching	Dense embeddings, multi-factor weighted scoring, evidence linking
Phase 7	AI Improvement Engine	Targeted bullet rewrites (X-Y-Z formula) without hallucination
Phase 8	Template Generation	Recruiter screening summaries, interview guides, shortlist memos
Phase 9	Recruiter Dashboard	Candidate ranking table, pipeline kanban, skill breakdown charts
Phase 10	Production Hardening	Rate limiting, Redis caching, structured logging, OpenAPI documentation
Phase 11	Testing & Verification	Unit test suite (PyTest + Jest), E2E test suite (Playwright)
Phase 12	Cloud Deployment	CI/CD GitHub Actions, Container registry, Cloud DB & Redis setup